

GUÍA DE LABORATORIO FORMATIVO — CLASE 17

Simulacro Oficial Evaluación Sumativa Hito 2: Proyecto AutoFix

Asignatura:	Programación Back End (TI3041) — 72 hrs presenciales
Unidad de Aprendizaje:	Unidad 2: Framework Back End (Django Admin & ORM Relacional)
Modalidad / Tiempo:	Taller Formativo Cronometrado (PLT) — 60 minutos individuales
Caso de Negocio:	AutoFix — Sistema de Gestión de Taller Mecánico y Órdenes de Reparación

1. Presentación del Caso de Negocio: AutoFix

El taller mecánico **AutoFix** requiere automatizar la recepción de vehículos y el seguimiento de sus órdenes de reparación. Para ello, necesita una plataforma web administrada con Django donde cada **Especialidad** (Mecánica General, Frenos y Suspensión, Electricidad, etc.) pueda gestionar sus **Órdenes de Reparación** de forma centralizada y eficiente.

OBJETIVO DEL SIMULACRO:

Construir de manera autónoma en 60 minutos el proyecto completo en Django con relaciones 1:N, personalizar el panel Django Admin con InlineModelAdmin, aplicar control de versiones en Git excluyendo .venv pero incluyendo db.sqlite3, e incluir la bitácora prompts_ia.md con el formato RTCF.

2. Requerimientos Técnicos y Paso a Paso

Etapa 1: Configuración del Proyecto y Modelos Relacionales (models.py)

- Crea el entorno virtual `.venv``, actívalo e instala Django (``pip install django``)
- Crea el proyecto ``autofix_project`` y la aplicación interna ``taller``. Regístrala en ``INSTALLED_APPS``.
- Define el modelo Padre ``Especialidad`` con:
 - ``nombre`` (CharField 100, único)
 - ``descripcion`` (TextField opcional)
 - ``activa`` (BooleanField, default True).
- Define el modelo Hijo ``OrdenReparacion`` con:
 - ``codigo_orden`` (CharField 10, único)
 - ``patente_vehiculo`` (CharField 8)
 - ``modelo_vehiculo`` (CharField 100)
 - ``cliente_nombre`` (CharField 150)
 - ``costo_estimado`` (IntegerField)
 - ``estado`` (CharField con choices: INGRESADO, EN_PROCESO, LISTO, ENTREGADO)
 - ``fecha_ingreso`` (DateField auto_now_add)
 - ``especialidad`` (ForeignKey a Especialidad, on_delete=models.CASCADE, related_name='ordenes').
- Ejecuta ``makemigrations`` y ``migrate`` para generar las tablas relacionales.

Etapa 2: Carga de Datos y Personalización Avanzada de Django Admin (admin.py)

- Crea un superusuario (``python manage.py createsuperuser``) e ingresa al panel en ``/admin/``.
- Poblar la base de datos con al menos 2 Especialidades (ej: 'Mecánica General', 'Frenos y Suspensión') y 6 Órdenes de Reparación asociadas.
- Crear la clase ``OrdenReparacionInline(admin.TabularInline)`` con ``model = OrdenReparacion`` y ``extra = 1``.
- Vincular el inline en ``EspecialidadAdmin`` (``inlines = [OrdenReparacionInline]``) y agregar un método ``@admin.display`` que muestre el total de órdenes asociadas por especialidad.
- Configurar ``OrdenReparacionAdmin`` con: ``list_display``, ``list_filter = ['estado', 'especialidad', 'fecha_ingreso']``, ``list_editable = ['estado']`` y ``search_fields = ['codigo_orden', 'patente_vehiculo', 'cliente_nombre', 'especialidad__nombre']``.
- Crear la acción masiva ``@admin.action`` llamada ``marcar_como_listo`` para actualizar el estado de múltiples órdenes seleccionadas a 'LISTO'.

Etapa 3: Control de Versiones Git y Documentación RTCF

- Inicializar repositorio local (`git init`).
- Crear el archivo `.gitignore` configurando la exclusión de `.venv/` y `\_\_pycache\_\_`. REGLA ACADÉMICA: Sí se debe incluir `db.sqlite3` para la evaluación.
- Generar el archivo de dependencias con `pip freeze > requirements.txt`.
- Crear la bitácora `prompts\_ia.md` registrando la asistencia de IA bajo el formato RTCF (Rol, Tarea, Contexto, Formato).
- Realizar el primer commit: `git add .` y `git commit -m "feat: Proyecto AutoFix con modelos relacionales y admin personalizado"`.

3. Pauta

Criterio	Indicadores de Logro Evaluados
1. Modelos 1:N	Define correctamente los modelos Especialidad y OrdenReparacion con ForeignKey, tipos de datos apropiados y reglas de integridad (on_delete=CASCADE).
2. Migraciones y BD	Ejecuta el ciclo de migraciones sin errores y puebla la base de datos con al menos 2 especialidades y 6 órdenes asociadas.
3. Admin & Inlines	Configura TabularInline en EspecialidadAdmin, búsquedas cruzadas (especialidad__nombre), filtros, list_editable y columna calculada.
4. Acciones Masivas	Implementa la acción personalizada @admin.action para actualizar masivamente el estado de órdenes seleccionadas a 'LISTO'.
5. Git & RTCF	Configura .gitignore excluyendo .venv e incluyendo db.sqlite3, genera requirements.txt y documenta la bitácora prompts_ia.md con formato RTCF.